



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Ingegneria
“Enzo Ferrari”

Fondamenti di elettronica

Esperienza di laboratorio 3:

Interrupts, Timers e Low-power modes

Dott. Tommaso Zanotti

email: tommaso.zanotti@unimore.it

Tipi di interazione tra CPU e periferiche

Polling (come fatto fino ad ora)

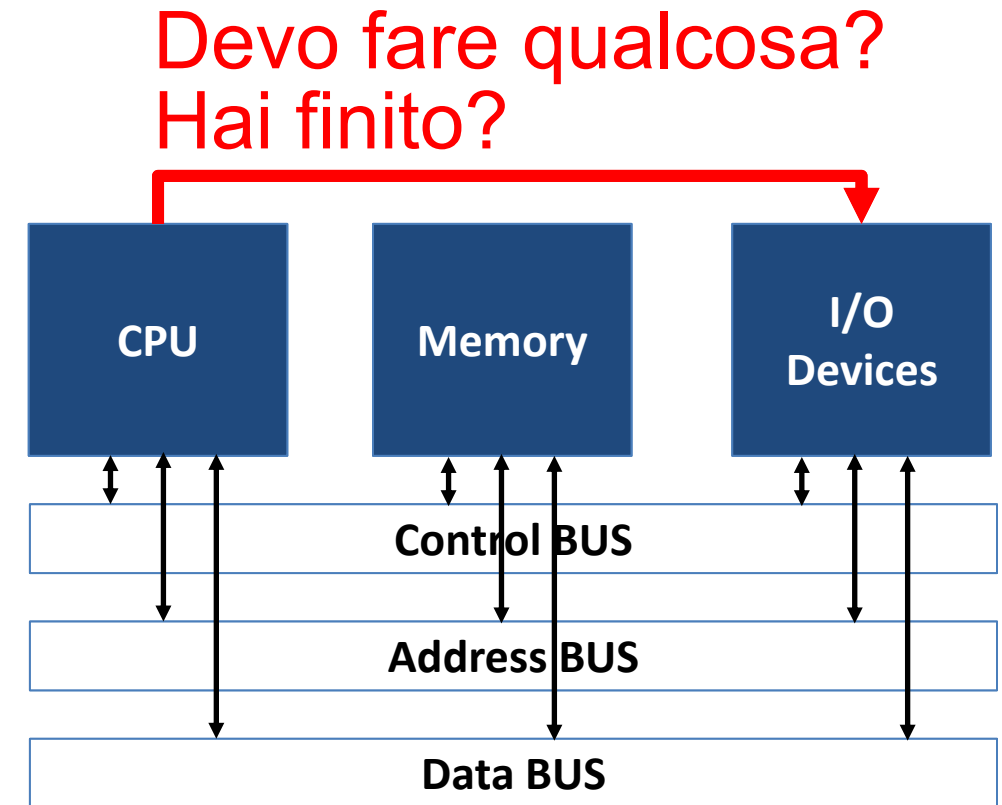
La CPU interroga attivamente e ciclicamente la periferica per verificare se ci siano nuovi dati acquisiti o se debba eseguire una certa azione.

Esempi:

- Esercitazione 1 - Pulsante
- Esercitazione 2 - Lettura sensori

Svantaggio: poco efficiente in alcuni casi poiché la CPU rimane in attesa di un certo evento. Esempio: avvio una conversione con un ADC ed attendo la fine della conversione

Vantaggio : codice più semplice



Polling: esempio

```
while (read_status(deviceID) == 1){ // 1 means device is busy
    // wait
}
```

Ad esempio se voglio scrivere dei dati su una memoria esterna prima di scrivere dei nuovi dati devo attendere di aver finito la scrittura precedente.

Vantaggi:

- La CPU è in pieno controllo
- Semplice

Svantaggi:

- La CPU deve continuamente interrogare la periferica per identificare cambiamenti del suo stato
- Il tempo usato per interrogare le periferiche è tempo di calcolo sottratto ad altri task

Tipi di interazione tra CPU e periferiche

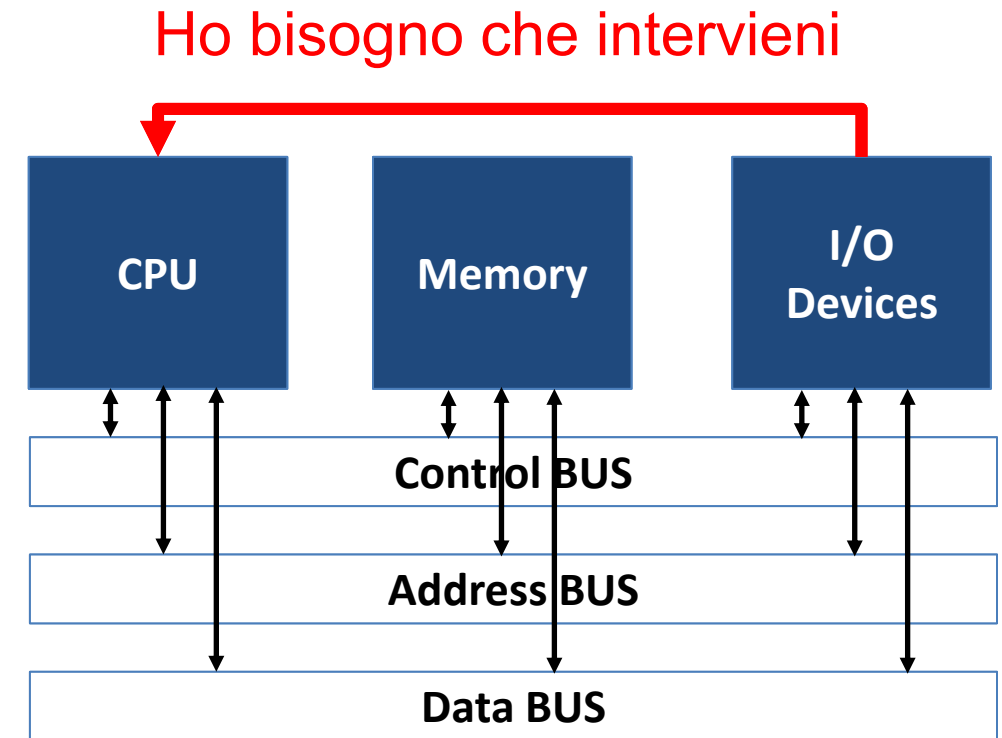
Interrupts

Le **interruzioni** (o **interrupts**) sono eventi che richiedono una risposta immediata da parte della CPU, interrompendo il flusso normale di esecuzione. Quando un'interruzione viene generata, il processore sospende temporaneamente l'esecuzione corrente, salva lo stato, e salta a una **Interrupt Service Routine (ISR)** o **Interrupt Handler**, dove viene gestito l'evento.

La periferica **avverte** la CPU dell'accadimento di un evento. La CPU può eseguire altre istruzioni fintanto che non sopraggiunge un evento che richieda il suo intervento diretto.

Vantaggi: più efficiente utilizzo delle risorse (il programma principale si interrompe soltanto quando è strettamente necessario)

Svantaggi: codice ed hardware più complessi

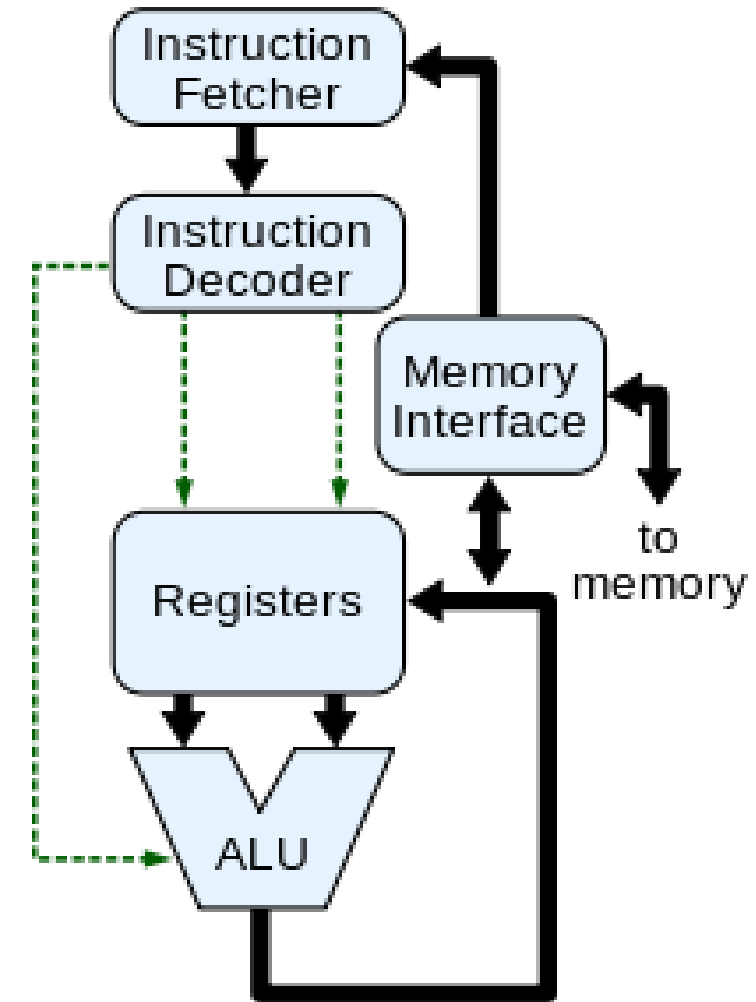


Tipi di interruzioni

Un segnale di **interrupt**, è un segnale asincrono che permette di **interrompere** il **normale ciclo di esecuzione del nostro programma**, ovvero la sequenza di istruzioni programmate, **per eventualmente svolgere un'altra azione**.

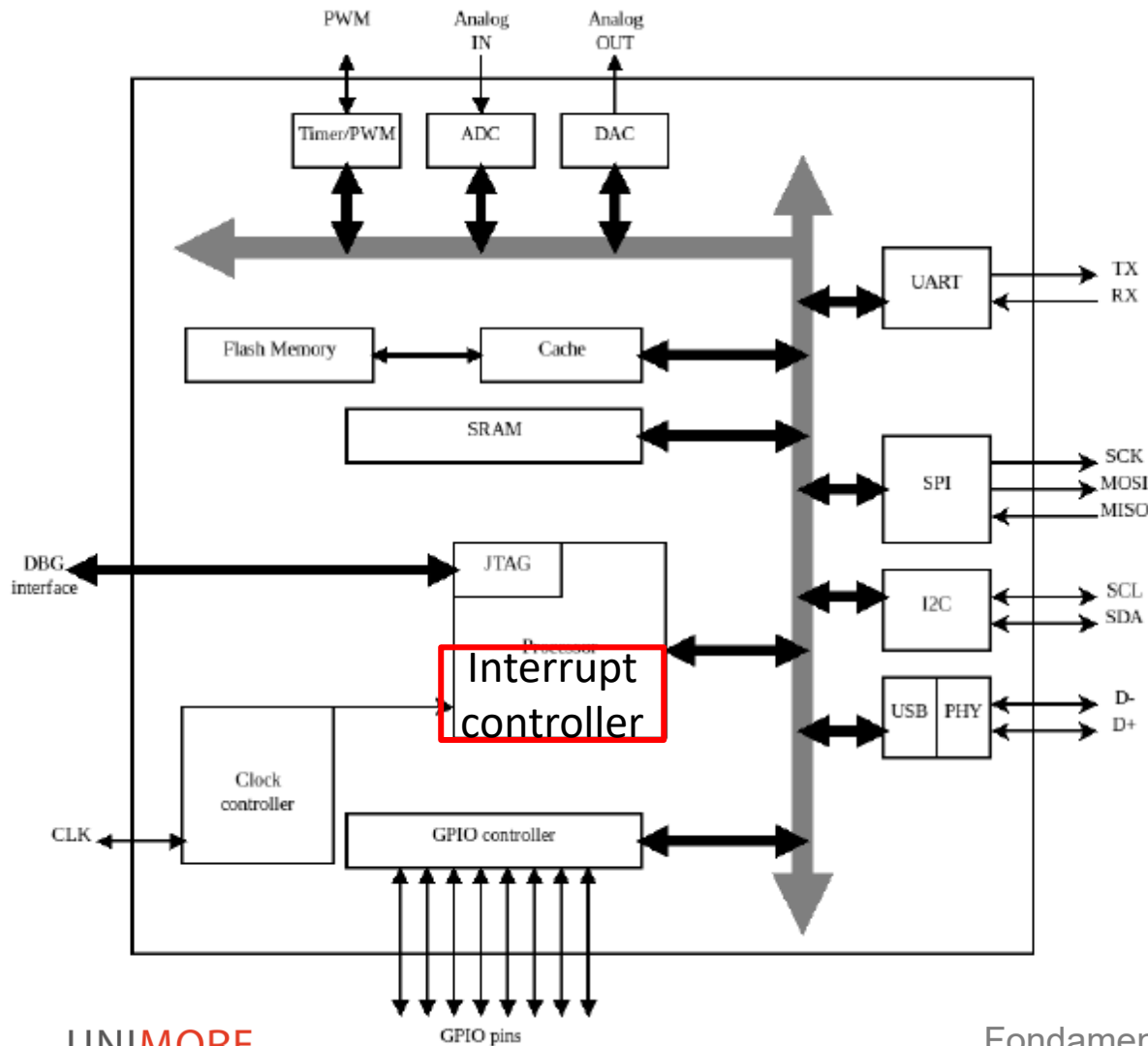
Tipicamente le interruzioni possono essere:

- **Esterne**, ad esempio:
 - Variazione di un pin GPIO da lato a basso o da basso ad alto
- **Interne**, ad esempio:
 - Scadenza di un timer
 - Trasferimento completato da una periferica come UART, I2C o SPI



Microcontrollori

Interrupt controller



- La CPU interrompe il regolare susseguirsi delle istruzioni per eseguire una routine specifica (*interrupt handler*) che risolve il bisogno indicato dalla richiesta di interruzione
- La gestione degli interrupt è fondamentale per lo sviluppo di sistemi **real-time**
- Ogni MCU contiene un **Nested Vector Interrupt Controller (NVIC)** che permette di:
 - Assegnare priorità alle interruzioni
 - Concatenare le interruzioni
- Tipicamente un NVIC supporta circa 240 sorgenti di interruzioni che possono essere raggruppate con fino a 256 livelli di priorità

Interrupt vector table

- Quando un'interruzione viene abilitata, il microcontrollore deve eseguire una specifica funzione
- All'interno della memoria del microcontrollore è presente un'area di memoria in cui è memorizzata la **interrupt vector table**, in cui in specifiche (e fisse) locazioni di memoria sono memorizzati gli indirizzi che portano all'area di memoria contenente le istruzioni da eseguire quando uno specifico segnale di interrupt viene generato
- Guardare a pag. 396 del **RM0351 Reference manual** (file en.DM00083560.pdf)

INTERRUPT VECTOR TABLE

RESET	INDIRIZZO DI INIZIO DEL PROGRAMMA
TIMER Interrupt	INDIRIZZO DI INIZIO DELLA SUBROUTINE DI INTERRUZIONE
.	
.	
.	
ADC-end of conversion	

MEMORIA

Timers

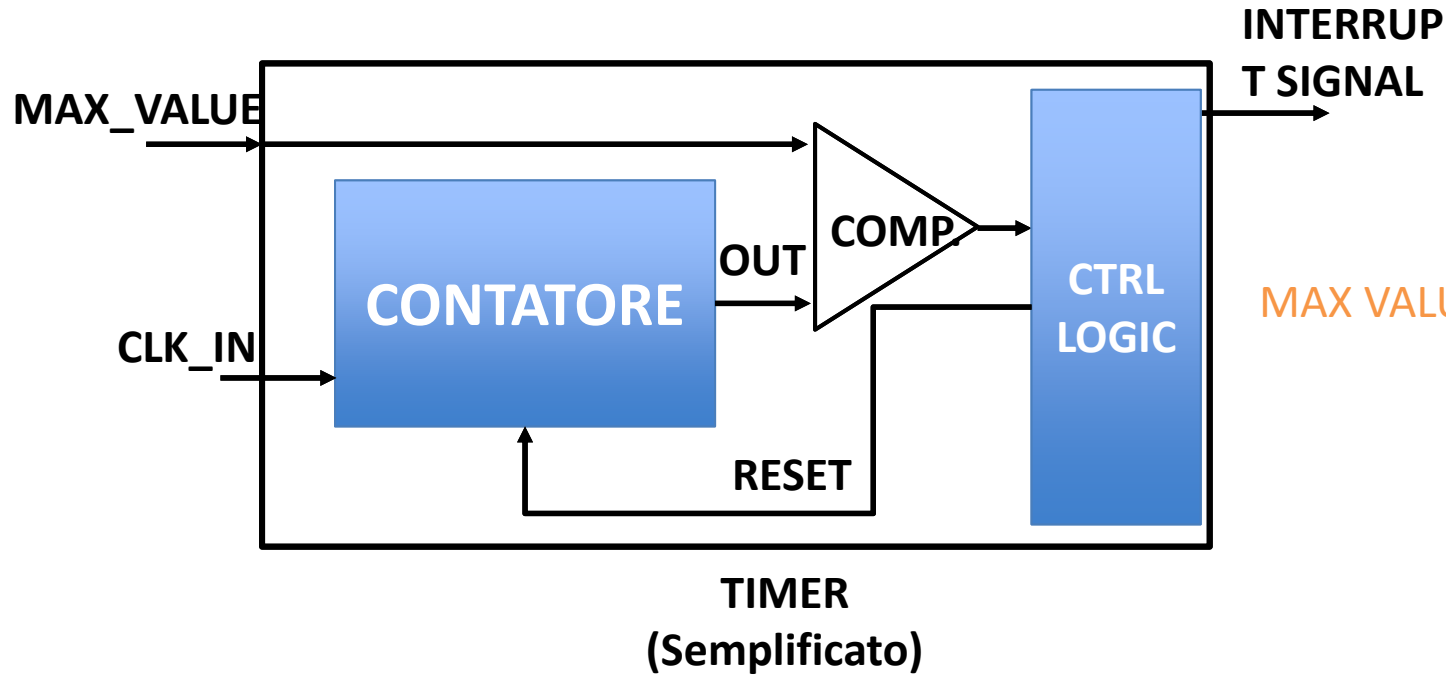
I timers

I Timers sono unità hardware che possono essere configurate per **contare cicli di clock e generare eventi periodici**. Sono utilizzati per eseguire operazioni come il calcolo del tempo, la generazione di PWM, o la temporizzazione di eventi. I microcontrollori STM32 includono diversi **timers general-purpose** e **timers avanzati**, ognuno con funzionalità specifiche.

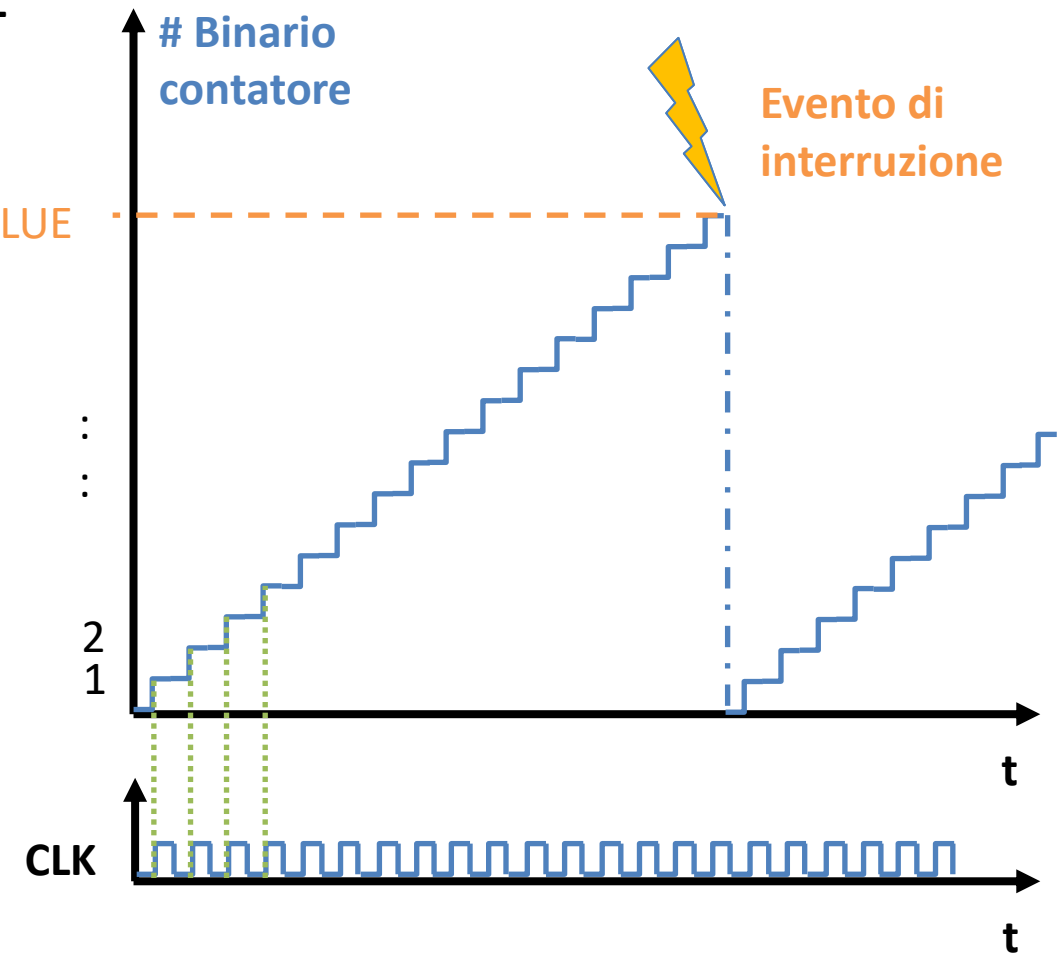
Supponiamo che si voglia configurare un timer per generare un'interruzione ogni 1 secondo.

1. Impostare il **prescaler** e il **periodo del timer**: Il prescaler divide il clock principale della CPU per rallentare il conteggio. Il periodo determina il numero massimo che il timer deve raggiungere prima di scatenare un'interruzione o un evento.
2. **Abilita l'interruzione del timer nel NVIC**: bisogna abilitare l'interruzione generata dal timer.
3. **Gestione dell'ISR per il timer**: Definire la routine di gestione dell'interruzione generata dal timer.

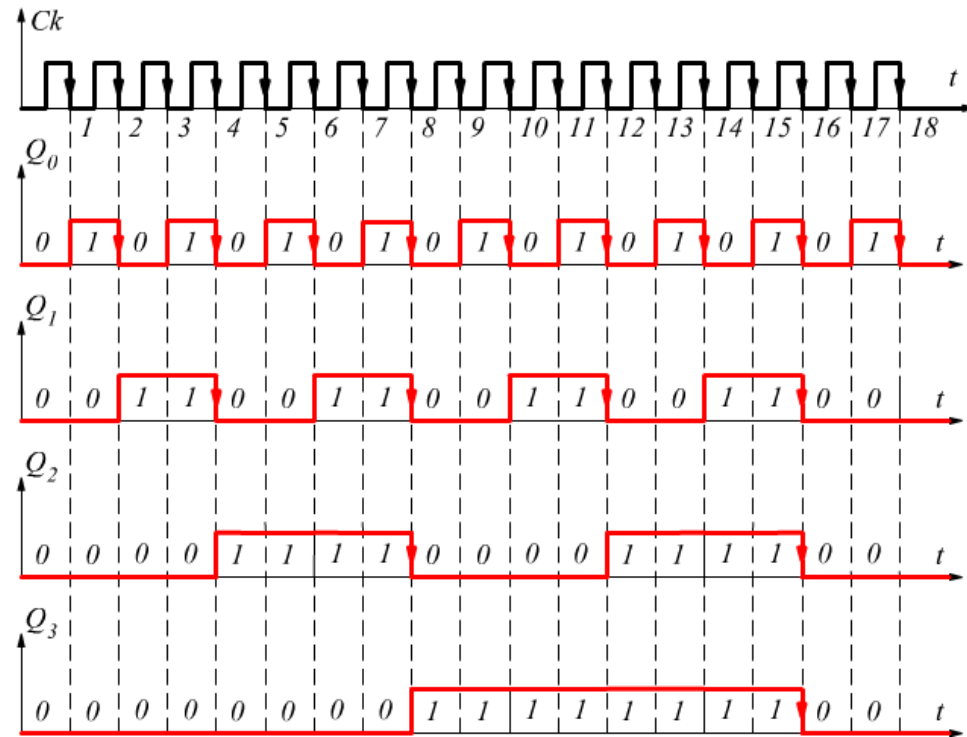
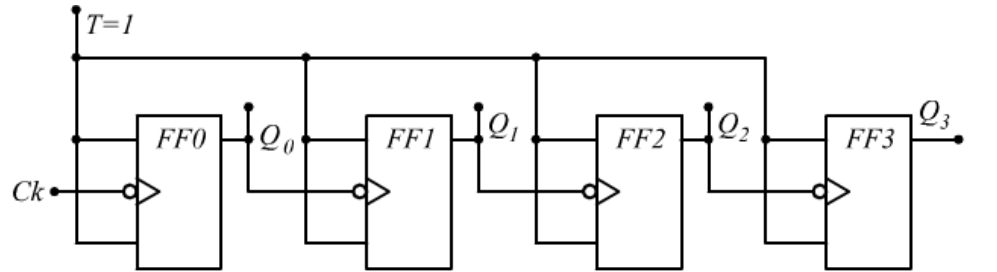
Interrupt generata da un timer



- MAX_VALUE definisce l'intervallo di tempo che passa tra un interrupt e la successiva
- Quando il contatore raggiunge MAX_VALUE il segnale di interrupt viene generato e passato come ingresso al gestore di interrupt ed il contatore resettato



Contatore digitale



- In figura, ad ogni fronte di discesa del segnale di clock in ingresso il numero binario rappresentato da Q3 Q2 Q1 Q0 viene incrementato di 1

Q3	Q2	Q1	Q0	# decimale
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	10
1	0	1	1	11
1	1	0	0	12
1	1	0	1	13
1	1	1	0	14
1	1	1	1	15

A seguito di una interruzione la periferica comunica con l'interrupt controller che esegue la specifica funzione di interrupt request handler (IRQHandler) all'interno della quale viene resettato il bit che ha causato l'interruzione ed esegue la funzione di callback.

Le funzioni di callback sono definite come *weak* nelle librerie HAL e quindi possono essere ridefinite nel nostro file *main.c*

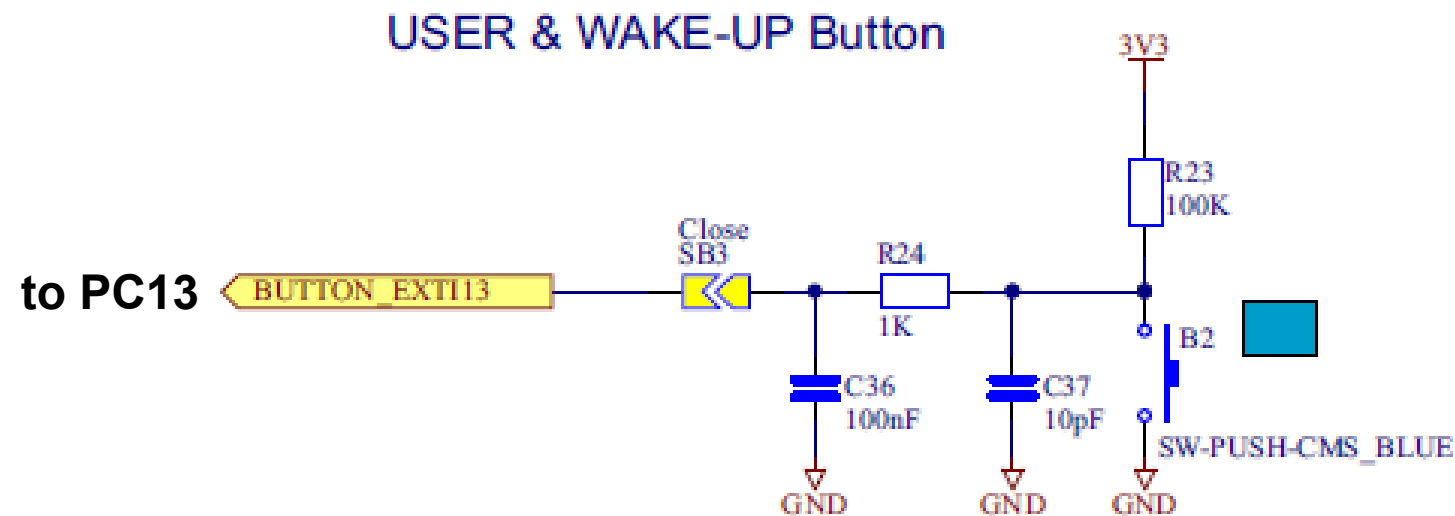
Progetto 1: Interruzioni generate da TIMER and READ BUTTON

Obiettivo: Rilevare quando il pulsante blu della scheda viene premuto tramite interrupt sul fronte di discesa. Ogni volta che il pulsante viene premuto vado a cambiare lo stato del LED1. Generare un'interruzione ogni secondo con un TIMER. Dentro la routine di interruzione del timer incrementare una variabile di conteggio ed inviarla al PC tramite la Virtual COM.

To do:

1. Configurare l'interruzione sul fronte di discesa del GPIO del pin collegato al pulsante blu
2. Identificare un timer appropriato e configurare l'interruzione
 - a. Scegliere un valore adatto per il prescaler
 - b. Scegliere il valore massimo del contatore in modo che l'interruzione sia generata ogni secondo
3. Configurare opportunamente la priorità tra le due interruzioni
4. Modificare le funzioni di callback ed il main

Pulsante a bordo della scheda di sviluppo



100nF should be place close to the MCU
10pF and 1K should be place close to the button

Low-power modes

Low-power modes

Dalla sezione 5.3 del manuale del microcontrollore:

«Several low-power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event. It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.»

Features

- Ultra-low-power with FlexPowerControl
 - 1.71 V to 3.6 V power supply
 - -40 °C to 85/105/125 °C temperature range
 - 300 nA in V_{BAT} mode: supply for RTC and 32x32-bit backup registers
 - 30 nA Shutdown mode (5 wakeup pins)
 - 120 nA Standby mode (5 wakeup pins)
 - 420 nA Standby mode with RTC
 - 1.1 μ A Stop 2 mode, 1.4 μ A with RTC
 - 100 μ A/MHz run mode
 - Batch acquisition mode (BAM)
 - 4 μ s wakeup from Stop mode
 - Brown out reset (BOR)
 - Interconnect matrix

Microcontrollore

Power Management and Energy Saving

- All'interno dei microcontrollori vengono implementate diverse **strategie volte a ridurre i consumi energetici**
- Il MCU può essere configurata dinamicamente per funzionare in specifiche **low-power modes** che:
 - Disabilitano le periferiche non utilizzate (dark silicon)
 - Riducono la frequenza di clock
 - Quest'ultimo viene distribuito solo alle periferiche necessarie

- **Normal operation mode:** Active components are selected through clock gating, and the clock is running at the desired frequency.
- **Sleep mode:** The CPU clock is temporarily suspended, but all the peripherals keep functioning as in normal mode. There is a noticeable, even if marginal, amount of power saved in this mode.
- **Stop mode:** The CPU clock and the bus clocks are disabled. All the peripherals powered by the microcontroller are off. The internal RAM and the CPU registers retain the stored values because the main voltage regulator stays on. The power consumption drops consistently.
- **Standby mode:** All the voltage regulators are off, and the content of the RAM and register is lost. A small amount of power, in the range of a few microwatts, may be required to keep the backup circuitry alive during the standby phase.

Low-power modes

In base alla modalità low-power configurata, solo alcune sorgenti sono in grado di risvegliare il microcontrollore

Table 22. Low-power mode summary

Mode name	Entry	Wakeup source ⁽¹⁾	Wakeup system clock	Effect on clocks	Voltage regulators	
					MR	LPR
Sleep (Sleep-now or Sleep-on-exit)	WFI or Return from ISR	Any interrupt	Same as before entering Sleep mode	CPU clock OFF no effect on other clocks or analog clock sources	ON	ON
	WFE	Wakeup event				
Low-power run	Set LPR bit	Clear LPR bit	Same as Low-power run clock	None	OFF	ON
Low-power sleep	Set LPR bit + WFI or Return from ISR	Any interrupt	Same as before entering Low-power sleep mode	CPU clock OFF no effect on other clocks or analog clock sources	OFF	ON
	Set LPR bit + WFE	Wakeup event			OFF	ON
Stop 0	LPMS="000" + SLEEPDEEP bit + WFI or Return from ISR or WFE	Any EXTI line (configured in the EXTI registers) Specific peripherals events	HSI16 when STOPWUCK=1 in RCC_CFGR MSI with the frequency before entering the Stop mode when STOPWUCK=0.	All clocks OFF except LSI and LSE	ON	ON
Stop 1	LPMS="001" + SLEEPDEEP bit + WFI or Return from ISR or WFE					
Stop 2	LPMS="010" + SLEEPDEEP bit + WFI or Return from ISR or WFE				OFF	
Standby with SRAM2	LPMS="011" + Set RRS bit + SLEEPDEEP bit + WFI or Return from ISR or WFE	WKUP pin edge, RTC event, external reset in NRST pin, IWDG reset	MSI from 1 MHz up to 8 MHz			
Standby	LPMS="011" + Clear RRS bit + SLEEPDEEP bit + WFI or Return from ISR or WFE	WKUP pin edge, RTC event, external reset in NRST pin, IWDG reset			OFF	OFF
Shutdown	LPMS="1--" + SLEEPDEEP bit + WFI or Return from ISR or WFE	WKUP pin edge, RTC event, external reset in NRST pin	MSI 4 MHz	All clocks OFF except LSE	OFF	OFF

Progetto 4: Test of the MCU low-power modes

Obiettivo: Provare le diverse modalità low-power (sleep, stop, standby) e capire come risvegliare il microcontrollore.

- Risvegliare il micro, sfruttando diverse sorgenti di interruzione (EXTI, USART RX)
- Evidenziare la differenza tra modalità «normale» ed «SleepOnExit»

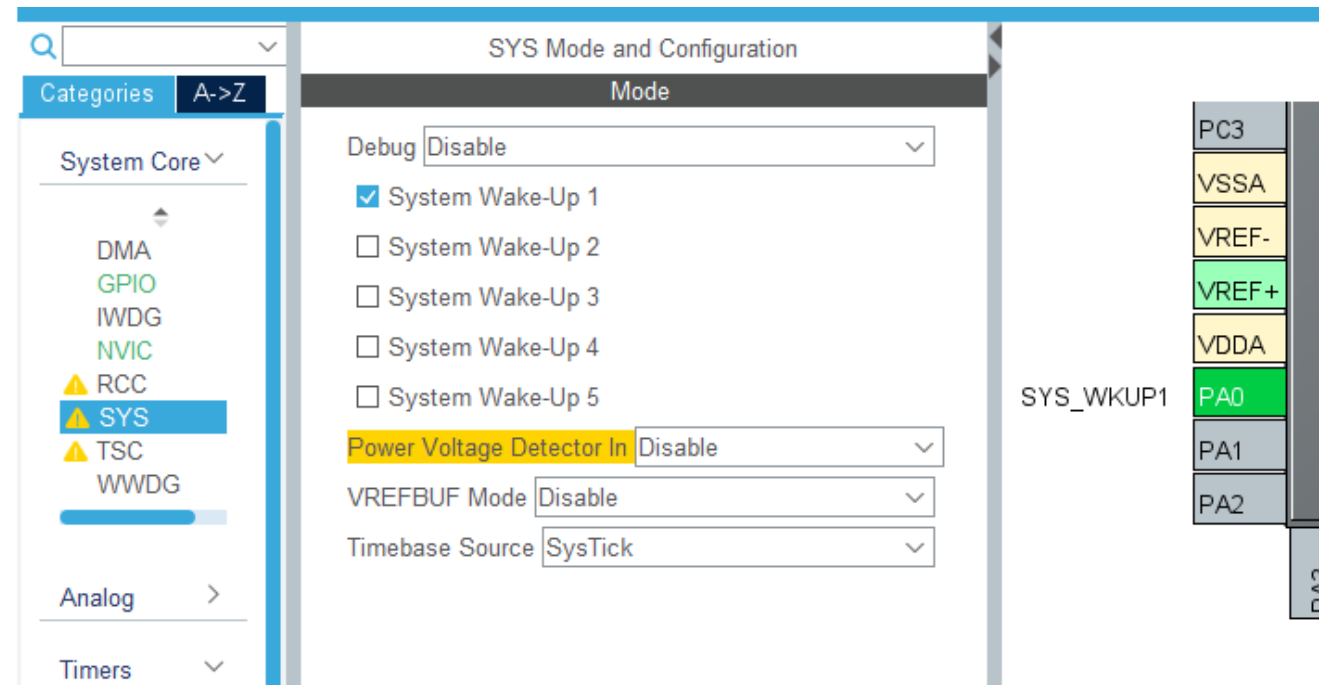
Progetto 4: Test of the MCU low-power modes

Informazioni aggiuntive

Wakeup da STANDBY con segnale di WAKEUP esterno

Occorre configurare il wakeup pin 1 come in figura

PC13 sarebbe collegato al wakeup pin 2, tuttavia per qualche motivo non funziona



Quindi per risvegliare il sistema occorrerà collegare a 3.3V il pin PA0

Progetto 4: Test of the MCU low-power modes

Informazioni aggiuntive

Sorgente di clock del modulo RTC (32 kHz)

